本科生毕业论文（设计）

开 题 报 告

题目： 基于 ESP32-S3 的智能终端设计与实现

学院： 工学院

专业： 电气工程及其自动化

学生姓名： 姚凯锐

学号： 2320158058

指导教师： 曹春妍

2026 年 11 月 16 日

一、选题依据（包括目的、意义、国内外现状和发展趋势，主要参考文献）

1.1 目的

本课题旨在设计并实现一台以 ESP32-S3 为核心的手持式智能终端，把“图形界面、联网信息服务、音频播放与语音交互、传感采集”等分散在多个独立实验中的功能，集成到一个可脱离电脑独立运行、由锂电池供电的实体设备上，形成从硬件接口、外设驱动、实时操作系统到上层应用的完整技术闭环。

通过本课题的研究与实现，一方面验证“单颗国产 MCU + 实时操作系统 + 开源图形库”这一技术组合在中小型智能终端产品中的可行性，探索在 320×240 分辨率、有限内存与算力条件下兼顾界面流畅度与功能完整性的工程方法；另一方面，把每一个外设封装为可复用、可测试的软件组件，沉淀出一套结构清晰、易于二次开发的嵌入式终端软件框架，为同类毕业设计、教学实验与小型产品原型提供可直接借鉴的实现路径与文档资料。

1.2 意义

技术意义。 传统单片机教学多停留在“点亮一个 LED、读取一个传感器”的裸机层面，外设之间彼此孤立，缺少多任务、图形界面与网络并存的系统级训练。本课题以 FreeRTOS 为调度核心、以 LVGL 为图形框架、以组件化方式组织驱动代码，需要处理任务优先级、栈空间、共享总线互斥、显示缓冲与刷新时序等一系列真实工程问题。研究并解决这些问题，有助于建立“面向产品而非面向实验”的嵌入式软件设计思维。

应用意义。 智能终端是物联网产业的重要形态，广泛应用于智能家居中控、便携信息终端、工业手持设备、教学实验平台等场景。ESP32-S3 作为国产芯片，集成 Wi-Fi 与蓝牙、内置向量指令并支持外部 PSRAM 扩展，整机物料成本低、开发工具链免费开源，适合作为低成本终端的核心方案。本课题实现的终端具备日历天气、音乐播放、语音交互、串口调试等功能，可直接用于家庭信息面板、实验室设备状态显示与嵌入式教学演示。

教学与个人发展意义。 课题覆盖原理图识读、引脚规划、驱动编写、GUI 设计、网络编程、音频处理与系统测试等完整链路，与电气工程及其自动化专业“软硬结合”的培养目标高度契合。通过独立完成课题，可系统提升嵌入式 C 语言编程

能力、硬件调试能力与工程文档写作能力，为从事嵌入式软件、物联网终端开发等工作打下基础。

1.3 国内外现状和发展趋势

（1）物联网终端与智能家居产业持续扩张。 近年来，全球物联网连接设备数量保持高速增长，据 IoT Analytics 统计，全球联网 IoT 设备数量已增长 14%、达到 211 亿台左右[3]；智能家居、可穿戴设备与工业手持终端成为主要增量市场。在国内，据 IDC 预测，2025 年中国智能家居设备市场出货量约 2.81 亿台，同比增长 7.8%，其中智能照明与中控屏类产品增长尤为明显[1]；中国信息通信研究院的测算显示，2025 年我国数字经济规模约 60 万亿元，其中产业数字化部分约 51 万亿元[2]，为智能终端的落地提供了庞大的产业土壤。用户对终端的期待也从“能联网”转变为“有屏幕、能触控、可语音”，这对终端的图形处理能力与人机交互设计提出了更高要求。

（2）主控芯片向“高集成 + 边缘智能”演进。 早期智能终端多采用“MCU + 独立 Wi-Fi 模组 + 显示驱动芯片”的多芯片方案，成本高、体积大、调试复杂。以 ESP32 系列为代表的 SoC 方案将射频、处理器与外设控制器集成于单芯片，显著降低了整机成本与设计难度；从市场层面看，2025 年全球 MCU 销售额约 326.2 亿美元，并预计在 2026 年增至 343.2 亿美元[4]，其中集成无线通信能力的 SoC 型 MCU 增长最快。ESP32-S3 在此基础上增加了用于神经网络加速的向量指令，并支持 Octal PSRAM 扩展，使终端在本地即可完成语音唤醒、关键词识别等轻量级边缘计算任务，代表了“端侧智能”的发展方向。乐鑫官方持续维护的 ESP-IDF 框架提供了完善的驱动库、网络协议栈与 FreeRTOS 集成，其 esp_lcd 组件已内置 ST7789 等常见屏幕驱动[5]，已成为该系列芯片事实上的标准开发框架。

（3）嵌入式图形界面从“自绘”走向“框架化”。 过去受限于内存与算力，嵌入式界面多以点阵字库加简单图形函数自行绘制，界面风格单一、代码复用性差。随着 PSRAM 的普及，开源图形库开始在 MCU 上广泛应用。LVGL 以其控件丰富、支持抗锯齿与动画、可裁剪、跨平台等特点成为主流选择，官方示例在约 50 KB RAM、100 KB Flash 的资源条件下即可运行，v9 版本在渲染架构与 API 上做了较大重构，进一步提升了刷新效率[6]。国内已有研究将 FreeRTOS 与 LVGL 结合，在 STM32 平台上实现工业控制人机界面[7]，或在智能家居控制系统中引入 LVGL 与

MQTT 完成本地界面与云端联动[8]，说明“RTOS + 开源 GUI”的技术路线在中低端硬件上已具备工程可行性。

（4）语音交互成为终端标配能力。 大模型与云端语音服务的成熟，使低成本终端也能获得自然语言交互能力。开源社区中，基于 ESP32-S3 的语音助手项目 xiaozhi-esp32 通过“PDM 麦克风采集—音频编码上传—云端识别与大模型对话—语音合成回放”的链路，实现了接近消费级产品的对话体验，并在社区中保持活跃更新[9]；已有研究进一步将 MQTT 与 ESP-AI 方案结合，构建具备语音控制能力的智能家居系统[10]，验证了在百元级硬件上构建语音终端的可行性。

（5）现有教学与研究案例的不足。 综观现有资料，多数公开教程与文献以“单点功能验证”为主：或侧重 ESP32 的联网与云平台接入[11,12]，或侧重无线通信模块本身的设计[13]，或聚焦于特定传感信号的采集与识别[14]；教学类教程则多停留在单个外设的驱动验证，缺少将显示、触摸、网络、音频、电源管理组织为一个完整终端的产品级工程案例。部分案例虽然功能齐全，但代码结构扁平、缺乏组件划分与任务规划，难以维护与扩展；此外，SPI 总线共享、显示刷新性能、触摸坐标标定、内存与任务栈规划等关键问题往往被略过，学习者遇到实际问题时缺乏方法指导。本课题正是针对上述不足，以“完整终端 + 分层架构 + 可复用组件”为目标展开设计。

（6）发展趋势。 综合来看，智能终端正沿着“更强的端侧算力、更自然的交互方式、更低的功耗、更完整的软件生态”四个方向演进：图形界面将由静态页面转向带动效的组件化界面；交互方式将由单一触控转向“触控 + 语音 + 传感”多模态融合；OTA 升级与设备管理将成为终端的基本能力；低功耗设计将从“能用电池”走向“长续航”。本课题在方案设计中有意为 OTA 升级、低功耗策略预留接口，使系统具备继续演进的空间。

1.4 主要参考文献

[1] IDC. 2025 年中国智能家居设备市场出货量预测[EB/OL]. (2024-12-18)[2026-11-08]. https://m.jrj.com.cn/madapter/24h/2024/12/18140146528681.shtml.
[2] 中国信息通信研究院. 中国数字经济发展研究报告（2025 年）[EB/OL]. [2026-11-08]. https://finance.eastmoney.com/a/202601153620278137.html.
[3] IoT Analytics. State of IoT: the number of connected IoT devices[EB/OL]. [2026-11-08]. https://iot-analytics.com/number-connected-iot-devices/.
[4] 中商产业研究院. 2026 年全球 MCU 市场规模及预测[EB/OL]. (2026-08-04)[2026-11-08]. https://www.askci.com/news/chanye/20260804/102019278581001850341402_4.shtml.

[5] 乐鑫信息科技. ESP-IDF 编程指南：LCD（esp_lcd 框架与 ST7789 驱动）[EB/OL]. [2026-11-08]. https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/peripherals/lcd/index.html.
[6] LVGL Ltd. LVGL: Light and Versatile Embedded UI Ecosystem[EB/OL]. [2026-11-08]. https://lvgl.io/.
[7] 刘瑾,邓文娟,汪志成,等. 基于 STM32 的工业控制 HMI 系统设计[J]. 仪表技术与传感器,2024(4).
[8] 郭书杰. 基于 STM32 的智能家居控制系统设计[J]. 微处理机,2024(4).
[9] 78. xiaozhi-esp32: 基于 ESP32-S3 的 AI 语音助手固件[EB/OL]. [2026-11-08]. https://github.com/78/xiaozhi-esp32.
[10] 冯天顺,王明星,刘帅,等. 基于 MQTT 与 ESP-AI 的智能家居系统[J]. 工业控制计算机,2026(6).
[11] 毛浩龙,艾红. 基于 ESP32 的智能家居项目设计与实现[J]. 工业仪表与自动化装置,2021(2):126-130+136.
[12] 潘亚宾. 基于 ESP32 和 ThingsBoard 云平台的智慧小屋系统设计[J]. 无线互联科技,2025(10).
[13] 高珂. 智能制造无线通信模块设计实现[J]. 中国科技投资,2026(13).
[14] 阳佩珉,闵华松. 嵌入式肌电腕带实时采集与识别系统设计[J]. 计算机工程,2025(2).

二、研究内容（具体研究/设计内容，重点解决的问题，预期结果）

2.1 具体设计内容

本课题设计并实现一台基于 ESP32-S3 的手持式智能终端。硬件以 ESP32-S3-WROOM-1-N16R8 模组为核心，该模组集成双核 Xtensa LX7 处理器（最高 240 MHz）、512 KB 片内 SRAM、8 MB Octal PSRAM 与 16 MB Flash，内置 2.4 GHz Wi-Fi 与低功耗蓝牙，能够同时承担界面渲染、外设驱动与网络通信任务。终端外扩 320×240 的 ST7789 SPI 彩色液晶屏与 XPT2046 电阻触摸控制器构成人机交互界面，配置三个独立按键与一个共阳极 RGB 指示灯作为辅助输入与状态指示；多媒体部分包含 Micro SD 卡座、NS4168 I2S 数字功放与 PDM 数字麦克风；电源部分由 TP4056 锂电池充电管理电路、拨动开关、分压采样电路与 ADC 检测电路组成，支持锂电池供电与 USB 充电。系统硬件资源与接口分配如表 1 所示。

表 1 系统硬件资源与接口分配

功能模块	主要器件	接口 / 引脚	说明
主控	ESP32-S3-WROOM-1-N16R8	—	双核 240 MHz，16 MB Flash，8 MB PSRAM，集成 Wi-Fi/BLE
显示	ST7789（320×240）	SPI2：CS=GPIO10、SCK=GPIO12、MOSI=GPIO11、DC=GPIO9、BL=GPIO14	RGB565 格式，40 MHz，背光支持 PWM 调光
触摸	XPT2046	SPI3：CS=GPIO7、CLK=GPIO15、DIN=GPIO6、	电阻式触摸，中断触发后查询坐标

		DO=GPIO5、IRQ=GPIO4	
存储	Micro SD 卡座	SPI3：CS=GPIO16、MOSI=GPIO17、MISO=GPIO8、SCK=GPIO18	FATFS 文件系统，与触摸共享 SPI3 总线
音频输出	NS4168 数字功放	I2S0：BCLK=GPIO41、LRC=GPIO42、DIN=GPIO40	单声道，44.1 kHz / 16 bit
音频输入	MSM261D3526H1CPM（PDM）	I2S0：CLK=GPIO38、DATA=GPIO39	16 kHz 单声道语音采集
指示	RGB 三色 LED	GPIO21 / GPIO47 / GPIO48	共阳极，低电平点亮，LEDC PWM 调光
按键	三个独立按键	GPIO2 / GPIO45 / GPIO46	内部上拉，低电平有效，软件消抖
电池检测	10 kΩ / 5.1 kΩ 分压	GPIO1（ADC1_CH0）	12 bit 采样，分压补偿系数 1.556
充电管理	TP4056	CHRG# / STDBY# 指示灯	红色为充电中，绿色为充满
调试串口	UART0	TXD=GPIO43、RXD=GPIO44	115200 8N1，用于日志与指令交互
程序下载	USB Type-C	GPIO19 / GPIO20	硬件自动进入下载模式

软件方面，终端采用 ESP-IDF v5.x 框架，以 FreeRTOS 为运行基础，按“硬件层—驱动层—中间件层—应用层”四层结构组织代码。驱动层将每个外设封装为独立组件，包括 rgb_set（LEDC 灯效）、key_set（按键扫描与消抖）、timer_set（通用定时器与呼吸灯）、adc_set（电池电压采样与校准）、uart_set（串口收发）、lcd_set（SPI 屏驱动）、touch_set（触摸坐标读取与标定）、lvgl_set（LVGL 移植与刷新）、ui_set（界面构建与事件注册）、wifi_set（联网与配网）、music_set（音乐播放）、xiaozhi_set（语音交互）与 task_set（任务划分与调度）；中间件层使用 LVGL v9 完成界面渲染，使用 FATFS 管理 SD 卡文件，使用 lwIP 提供网络协议栈，使用 NVS 保存网络凭据与用户参数；应用层在统一桌面框架下实现设置、日历天气、音乐、AI 语音、串口终端、小游戏与关于等应用。系统主要功能结构如图 1 所示。



图 1 系统主要功能结构图

具体功能设计如下：

1. 图形用户界面：基于 LVGL 构建多页面界面，统一配色、字体与图标规范，实现桌面主界面与应用页面切换、页面进出动画与状态栏显示，界面刷新率不低于 30 fps。

2. 触摸与按键交互：读取 XPT2046 触摸坐标并完成方向映射与边界标定，实现按钮点击、列表滚动与页面滑动；三个按键支持短按、长按识别，用于返回桌面、调节亮度与快速唤醒。

3. 网络与信息服务：以 STA 模式接入 Wi-Fi，支持通过屏幕软键盘输入账号密码完成配网；联网后通过 HTTP 接口获取实时天气，通过 SNTP 完成网络校时，并在日历页面显示日期、星期与节日信息。

4. 音乐播放：从 Micro SD 卡读取音频文件，解码后经 I2S 输出至 NS4168 功放，实现播放、暂停、切换与音量调节，界面显示曲名、进度与频谱动效。

5. AI 语音交互：通过 PDM 麦克风采集语音，编码后经 Wi-Fi 上传至语音服务完成识别与大模型对话，再将合成语音经 I2S 播放，形成“按键唤醒—说话—应答”闭环。

6. 串口终端：在屏幕上实现串口调试终端，支持软键盘输入、数据发送与接收显示，可作为调试工具对下位机进行指令下发与日志查看。

7. 电量监测与电源管理：ADC 采集电池分压电压并换算实际电压，映射为电量百分比显示于状态栏；结合充电指示灯提示充电状态，低电量时告警并降低屏幕亮度。

8. 状态指示灯效：基于 LEDC 输出 PWM 驱动共阳极 RGB 灯，实现呼吸灯、颜色渐变与状态色指示，PWM 分辨率不低于 10 位，颜色过渡平滑。

9. 系统设置与扩展：提供亮度、音量、Wi-Fi 信息、系统版本与硬件信息等设置项，参数保存于 NVS 掉电不丢失；移植 LVGL 小游戏与“关于”页面，预留 OTA 升级接口。

2.2 重点解决的问题

1. **多外设共享总线与时序冲突问题。** 触摸控制器与 SD 卡共用 SPI3 总线，二者分属不同任务、访问频率差异大，若不加互斥会出现数据错乱；LCD 独占 SPI2

并以 40 MHz 高速传输，刷屏时会长时间占用 CPU 与总线。拟通过为 SPI 设备分别封装独立的 CS 控制与总线互斥锁、将刷屏操作集中到单一任务、必要时使用 DMA 传输等方式解决。

2. **LVGL 在 SPI 屏上的刷新性能问题。** 320×240、RGB565 格式下一帧数据量为 150 KB，若采用全屏刷新且不使用缓冲，帧率与响应速度均无法满足要求。拟采用“部分缓冲 + 局部刷新”策略，按屏幕区域计算脏矩形，仅刷新变化区域；同时正确配置 LVGL 的显示缓冲大小、刷新周期与 tick 来源，并处理 ST7789 在 SPI 传输下的 RGB565 字节序（必要时开启字节交换），确保颜色正确且无明显撕裂。

3. **电阻触摸的坐标标定与抖动问题。** XPT2046 输出为原始 ADC 值，存在方向、偏移与非线性误差，且触摸瞬间抖动较大。拟采用“采集多次取中值 + 阈值滤波 + 松手确认”的流程，通过四点标定法计算坐标映射系数，并结合中断触发与轮询相结合的方式兼顾响应速度与 CPU 占用。

4. **FreeRTOS 任务划分与内存规划问题。** 界面刷新、触摸采集、网络请求、音频播放与语音上传彼此独立且实时性要求不同，若全部放在一个任务中会导致界面卡顿。拟按“界面任务（较高优先级、周期刷新）—输入任务（中断 + 队列）—网络任务（阻塞等待）—音频任务（流式回调）”划分，明确任务优先级、栈大小与通信队列，配合 8 MB PSRAM 为 LVGL 缓冲与音频缓冲分配内存，并通过栈水位监测与内存泄漏检查验证稳定性。

5. **音频录放链路的一致性问题。** 播放链路为“文件—解码—I2S 输出—功放”，录音链路为“麦克风—PDM 采集—编码—上传”，两条链路共用 I2S0 外设，采样率（44.1 kHz 与 16 kHz）、位深与声道配置不同。拟将两条链路在不同阶段分别初始化与释放，避免时钟与 DMA 缓冲冲突；同时处理音频数据的分包、缓冲欠载与网络抖动，减少播放卡顿与录音丢帧。

6. **联网稳定性与信息安全问题。** 无线信号波动会导致请求超时与连接断开，Wi-Fi 账号密码若硬编码在源码中则会随代码外泄。拟实现断线自动重连与请求超时重试机制，将网络凭据与用户参数保存于 NVS，避免在仓库中提交含明文密码的配置文件；对外部接口返回的数据做长度与格式校验，防止缓冲区越界。

7. **电池电量检测的准确性问题。** 锂电池电压与剩余电量呈非线性关系，且 ADC 存在个体差异与噪声。拟采用多次采样取平均、按分压比（10 kΩ/5.1 kΩ）与

补偿系数换算真实电压，结合分段线性映射估算电量百分比，并在低电量时提前告警，避免过放损伤电池。

2.3 预期结果

1. 完成一台可独立运行的手持式智能终端样机，实现图形界面、触摸与按键交互、联网信息服务、音乐播放、语音交互、串口终端、电量监测与灯效指示等全部预定功能。

2. 形成结构清晰、组件化划分的固件工程：驱动层组件接口统一、可单独测试；应用层通过事件机制解耦；工程可通过 ESP-IDF 一条命令完成编译、烧录与监控。

3. 界面刷新率达到 30 fps 以上，触摸响应延迟低于 100 ms，页面切换与列表滑动流畅无卡顿；系统连续运行 72 小时不出现死机、花屏与触摸失灵。

4. 终端支持锂电池供电，正常使用条件下续航不少于 4 小时，电量显示误差控制在 ±5% 以内，低电量可正确告警。

5. 完成毕业论文撰写，整理出硬件接口说明、软件架构说明、关键问题解决方案与测试记录，形成可复用的开发文档与演示材料。

三、研究/设计方案（包括不少于三个研究/设计方法、技术路线，理论分析、计算、实验方法和步骤及其可行性，可能出现的技术问题及解决办法）

3.1 设计方法

1. **需求分析法。** 从使用场景出发梳理功能清单，区分“必须具备”与“可以扩展”的功能；同时明确非功能指标，如界面帧率、触摸延迟、续航时间、连续运行时长与内存占用上限，作为后续设计与测试的验收依据。

2. **分层架构设计法。** 将系统划分为硬件层、驱动层、中间件层与应用层，层间只通过约定接口通信：驱动层向上提供与硬件无关的函数（如 lcd_set_draw_bitmap、adc_set_get_voltage），应用层不直接操作寄存器。分层使各模块可独立开发、替换与测试，也便于后期移植到其他开发板。

3. **组件化设计法。** 每个外设对应一个 ESP-IDF 组件，包含源文件、头文件与 CMakeLists.txt，通过 CMake 依赖声明被主程序引用；组件内部隐藏实现细节，对外只暴露必要接口，避免全局变量与头文件交叉包含。

4. **技术选型与对比法。** 对关键环节给出候选方案并说明取舍：开发框架在 Arduino 与 ESP-IDF 之间选择后者，理由是官方维护、组件化、调试能力强；图形库在 LVGL 与自绘之间选择 LVGL，理由是控件完备、支持动画与抗锯齿；文件系统选择 FATFS 以兼容 SD 卡；网络协议选择 lwIP 内置的 HTTP 客户端与 SNTP 客户端，减少第三方依赖。

5. **硬件接口核对与引脚规划法。** 依据开发板原理图逐个核对网络名、电平极性（如 RGB 灯为低电平点亮、按键为低电平有效）与默认状态，形成引脚分配表；对共用总线的器件标注片选与共享关系，对未连接或悬空的引脚明确处理方式，避免“照抄教程”导致的驱动失败。

6. **实时性与资源计算法。** 按帧率目标反推刷屏带宽需求：320×240×2 B ≈ 150 KB/帧，30 fps 时约 4.5 MB/s，SPI 在 40 MHz 下单向理论带宽约 5 MB/s，因此必须采用局部刷新而非全屏刷新；按 LVGL 缓冲与音频缓冲需求估算 PSRAM 占用，为任务栈与堆预留余量。

7. **实验验证与迭代法。** 采用“最小系统—单外设—多外设—整机”的渐进式验证路线：先验证编译烧录与串口日志，再逐个验证 GPIO、PWM、ADC、UART、SPI 外设，随后集成显示与触摸，最后接入网络与音频，每一步均保留可回退的版本，便于定位问题。

8. **可靠性与安全性设计法。** 为关键任务启用看门狗；对指针、数组下标与数据长度做边界检查；对网络请求设置超时与重试；敏感参数存入 NVS 而非源码；异常时输出带标签的日志（ESP_LOG），便于现场定位。

9. **用户体验设计法。** 界面遵循“一眼看懂”原则：图标语义直观、层级不超过三级、常用功能一屏可达；统一动效时长与缓动曲线；对耗时操作给出加载提示；触摸目标不小于 40×40 像素，避免误触。

10. **测试与文档化方法。** 制定功能测试用例表，逐项记录预期结果与实际结果；对稳定性、续航与内存占用做专项测试；同步编写接口说明、问题记录与使用手册，使成果可复现、可移交。

3.2 技术路线

1. **需求分析与指标确定。** 明确终端定位、功能清单与性能指标，形成需求规格说明，作为后续设计与验收的依据。

2. **硬件方案核对。** 依据原理图核对主控型号、存储容量、外设接口与电平极性，建立引脚分配表，确认 SPI2/SPI3/I2S0/UART0/ADC 等资源的使用关系。

3. **开发环境搭建。** 安装 VSCode、ESP-IDF 工具链与串口监视工具，建立 Git 仓库与工程骨架，完成 Hello World 工程的编译、烧录与日志输出验证。

4. **基础外设驱动开发。** 依次实现并验证 GPIO 输出（RGB 灯）、GPIO 输入（按键）、LEDC PWM 调光、通用定时器（呼吸灯）、ADC 采样（电池电压）与 UART 收发，形成 rgb_set、key_set、timer_set、adc_set、uart_set 等组件。

5. **显示与触摸子系统。** 基于 esp_lcd 组件驱动 ST7789，实现清屏、字符与图形显示、图片绘制；实现 XPT2046 触摸坐标读取、方向映射与四点标定，完成 touch_set 组件。

6. **LVGL 移植与界面框架。** 配置 LVGL 的显示缓冲、刷新回调与 tick 来源，完成 lvgl_set 组件；建立多页面 UI 框架（桌面、设置、日历天气、音乐、AI 语音、串口终端、小游戏、关于），实现页面跳转与事件回调。

7. **多任务架构搭建。** 按职责划分 FreeRTOS 任务并设定优先级与栈大小，建立任务间队列与事件通知机制；将界面刷新、输入处理、网络请求与音频播放解耦，避免相互阻塞。

8. **网络与信息服务。** 实现 Wi-Fi STA 连接、断线重连与屏幕配网；接入 HTTP 天气接口与 SNTP 校时服务；完成日历天气应用的数据解析与界面刷新。

9. **音频录放链路。** 挂载 SD 卡并读取音频文件，完成解码与 I2S 输出；配置 PDM 麦克风采集与编码上传，实现语音交互应用；完成音量控制与播放状态显示。

10. **系统集成与优化。** 联调各模块，解决总线冲突、内存占用与刷新性能问题；优化界面动效、触摸响应与功耗表现；完成看门狗、异常处理与参数持久化。

11. **测试、文档与成果整理。** 按测试用例逐项验证功能，开展稳定性、续航与内存测试；整理代码仓库、接口文档、测试记录与论文材料，准备答辩演示。

开发过程中采用的总技术路线与阶段划分如图 2 所示，系统总体软件架构如图 3 所示。



图 2 系统开发思维导图

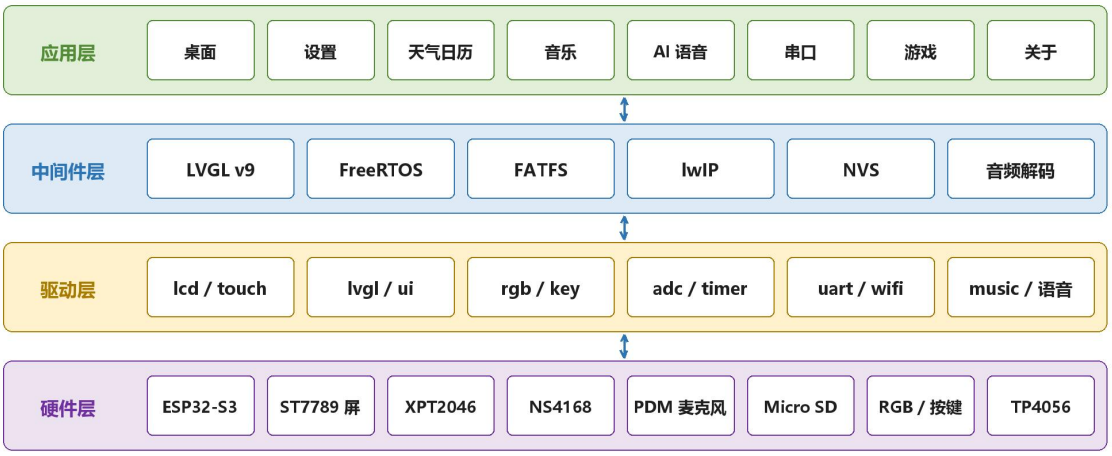


图 3 系统总体软件架构图

可能出现的技术问题及解决办法：

1. 屏幕出现花屏、颜色异常或显示偏移。先检查 SPI 时钟频率与 DC/CS 时序，再核对 ST7789 的初始化命令与偏移参数；颜色异常多由 RGB565 字节序引起，可通过开启 LVGL 的颜色字节交换或调整发送顺序解决。

2. 触摸坐标与实际点击位置不一致。通过四点标定建立原始值与屏幕坐标的线性映射，必要时区分横竖屏的 swap/mirror 设置；对边缘区域做钳位处理。

3. 系统运行一段时间后死机或重启。优先排查任务栈溢出与内存泄漏，使用栈水位查询与堆信息打印定位；确认中断服务函数中未调用阻塞 API，并检查看门狗喂狗逻辑。

4. 音频播放卡顿或杂音。检查 I2S 采样率与功放配置是否匹配、DMA 缓冲是否欠载、SD 卡读取速度是否足够；适当增大缓冲并降低解码线程的阻塞时间。

5. Wi-Fi 连接不稳定或请求超时。增加重连机制与超时重试，避免在网络任务中进行长时间阻塞操作；必要时降低刷新频率，为网络任务让出 CPU。

6. 电量显示跳变。对 ADC 采样值做滑动平均滤波，按锂电池放电曲线分段映射，并设置最小变化阈值，避免数值频繁跳动。

四、研究计划与进度安排

第 1 阶段：需求分析与方案确定（8 月 1 日—8 月 31 日）

8 月 1 日—8 月 7 日：调研同类智能终端产品与开源方案，明确终端定位、目标使用场景与核心功能清单。

8 月 8 日—8 月 14 日：细化功能需求与非功能需求（界面帧率、触摸延迟、续航、稳定性等指标），完成需求规格说明。

8 月 15 日—8 月 21 日：对照开发板原理图核对主控资源与外设接口，完成引脚分配与硬件方案确认。

8 月 22 日—8 月 31 日：整理需求与硬件分析结果，制定总体方案与关键里程碑时间表。

第 2 阶段：系统设计（9 月 1 日—9 月 14 日）

9 月 1 日—9 月 7 日：完成软件总体架构设计，划分硬件层、驱动层、中间件层与应用层，确定组件接口规范。

9 月 8 日—9 月 14 日：完成技术选型（ESP-IDF 版本、LVGL 版本、文件系统与网络方案），设计数据结构与任务划分方案。

第 3 阶段：环境搭建、原型设计及任务书撰写（9 月 15 日—9 月 30 日）

9 月 15 日—9 月 20 日：搭建开发环境，安装 VSCode、ESP-IDF 工具链与串口调试工具，配置 Git 版本管理。

9 月 21 日—9 月 26 日：完成点亮 RGB、按键读取、串口打印等最小系统验证，打通编译、烧录与日志链路。

9 月 27 日—9 月 30 日：撰写任务书，确定论文提纲与开发计划。

第 4 阶段：驱动开发及开题报告撰写（10 月 1 日—10 月 31 日）

10 月 1 日—10 月 10 日：完成 GPIO、PWM、定时器、ADC、UART 等基础外设驱动组件的封装与测试。

10 月 1 日—10 月 15 日：撰写开题报告，完成文献调研与方案论证。

10 月 11 日—10 月 20 日：完成 SPI LCD 显示驱动与 XPT2046 触摸驱动，实现字符与图形显示、触摸坐标标定。

10 月 21 日—10 月 31 日：完成 LVGL 图形库移植，实现界面显示、触摸交互与基础页面布局。

第 5 阶段：应用功能开发（11 月 1 日—11 月 30 日）

11 月 1 日—11 月 10 日：完成界面框架与事件机制，实现桌面、设置、关于等页面及页面切换逻辑。

11 月 11 日—11 月 20 日：完成 Wi-Fi 联网、天气获取、网络校时与日历天气应用；完成 FreeRTOS 多任务划分与串口终端应用。

11 月 21 日—11 月 30 日：完成 SD 卡挂载、音乐播放与 AI 语音交互功能，实现音频录制与播放链路。

第 6 阶段：系统集成与测试（12 月 1 日—12 月 10 日）

12 月 1 日—12 月 5 日：完成各功能模块联调，解决总线冲突、内存占用与任务调度问题。

12 月 6 日—12 月 10 日：开展功能测试、稳定性测试与续航测试，记录并修复缺陷。

12 月 1 日—12 月 30 日：完成毕业论文撰写与修改。

第 7 阶段：修复与优化（12 月 11 日—12 月 15 日）

12 月 11 日—12 月 13 日：修复测试阶段发现的问题，完善异常处理与容错逻辑。

12 月 14 日—12 月 15 日：优化界面刷新性能、触摸响应与功耗表现，提升整体使用体验。

第 8 阶段：部署与答辩准备（12 月 16 日—12 月 20 日）

12 月 16 日—12 月 18 日：整理工程代码与文档，制作演示固件与答辩材料。

12 月 19 日：完成系统最终版本固化，整理测试报告与使用说明。

12 月 20 日：提交毕业论文与全部材料，准备答辩演示。

各阶段的进度安排如图 4 所示。

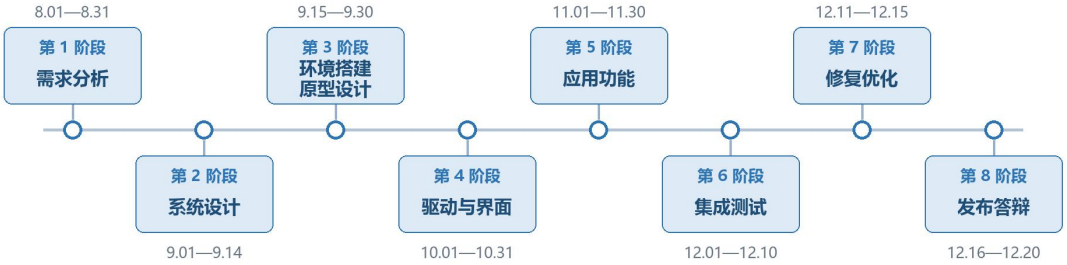


图 4 毕业设计阶段进度安排

五、预期达到的效果与存在的主要困难

5.1 预期达到的效果

1. 完成一台功能完整、可脱离电脑独立运行的手持式智能终端，实现图形界面、触摸与按键交互、联网信息服务、音乐播放、语音交互、串口调试、电量监测与灯效指示等预定功能，各项功能在实物上稳定运行。

2. 形成一套结构清晰、易于扩展的嵌入式终端软件框架：外设驱动组件化、界面与应用解耦、任务职责明确，后续增加新应用或更换屏幕、传感器时，只需替换对应组件而无需改动整体架构。

3. 界面刷新率达到 30 fps 以上，触摸响应延迟低于 100 ms，系统连续运行 72 小时稳定无异常，验证分层架构与多任务方案在资源受限平台上的有效性。

4. 完成一份内容完整、数据翔实的毕业论文，系统阐述终端的需求分析、方案设计、关键问题解决过程与测试结果，并整理出可复用的开发文档与演示材料。

5. 通过本课题的实践，系统掌握 ESP-IDF 开发流程、LVGL 界面开发、FreeRTOS 多任务设计与硬件调试方法，具备独立完成中小型嵌入式终端项目的能力。

5.2 存在的主要困难

1. **多外设并发带来的调试复杂度。** 显示、触摸、SD 卡、音频与网络同时运行时，问题现象往往相互掩盖（如界面卡顿可能由刷屏、内存不足或任务抢占引起），定位需要结合日志、栈水位与总线波形综合分析，对调试经验要求较高。

2. **资源受限条件下的性能平衡。** 屏幕刷新、音频缓冲与网络收发都需要占用内存与 CPU，而任务栈、PSRAM 与带宽总量有限，如何在保证界面流畅的同时不造成音频卡顿与网络超时，需要在实测中反复调整缓冲大小、任务优先级与刷新策略。

3. **音频与语音链路的实现难度。** 涉及 I2S 配置、PDM 采集、音频编解码、网络传输与云端协议对接，环节多、依赖外部服务，任一处参数不匹配都会导致无声、杂音或对话失败，需要逐段验证并准备降级方案。

4. **时间与硬件条件的限制。** 课题周期内需同时完成硬件调试、软件开发、测试与论文撰写，开发板与外围器件若出现故障将直接影响进度；部分云端接口存在调用频率与网络环境限制，可能影响联调效率，需提前准备备用方案与离线演示版本。

指导教师意见（教师可根据研究的理论意义和应用价值、创新之处、研究方法可行性等，提出不少于两方面的指导意见）：

指导教师签名：　　2026 年 11 月 17 日

专业负责人意见：

专业负责人签名：　　2026 年 11 月 22 日

所在院系意见：

院系负责人签名：　　2026 年 11 月 30 日